

Systèmes de Recommandation (RecSys)

Principes Fondamentaux, Algorithmes de Filtrage, Factorisation Matricielle & Évaluation

1. Introduction et Enjeux

Les **systèmes de recommandation** sont des algorithmes d'intelligence artificielle conçus pour prédire l'intérêt ou la note qu'un utilisateur attribuerait à un élément (produit, film, article) afin de filtrer la surcharge d'information (*Information Overload*).

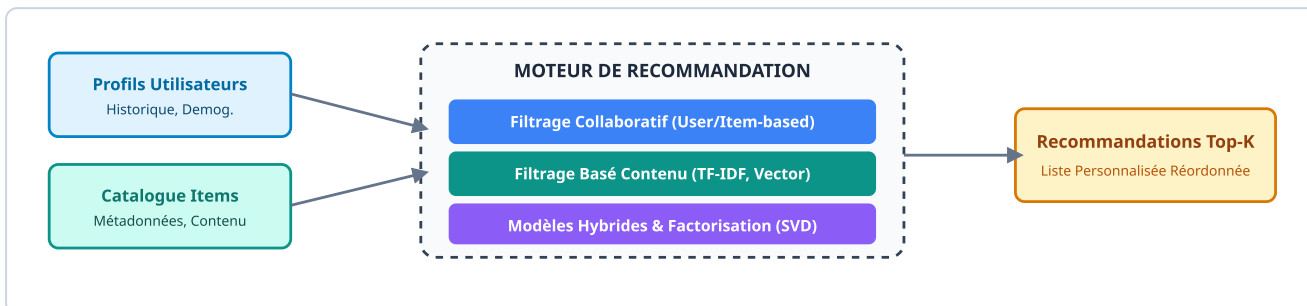
Données Explicites vs Implicites

- Explicites** : Notes (1 à 5 étoiles), likes, avis. Directes et précises mais rares (matrice très creuse).
- Implicites** : Clics, temps de visionnage, historique d'achats, clics. Abondantes mais bruitées.

Objectif Algorithmique

Construire une fonction d'appariement $f: U \times I \rightarrow \mathbb{R}$ prédisant l'appétence de l'utilisateur $u \in U$ pour l'item $i \in I$, afin de délivrer un classement ordonné (*Top-N Recommendation*).

2. Vue d'Ensemble des Architectures



3. Typologie des Approches

Collaboratif

Filtrage Collaboratif

S'appuie exclusivement sur les interactions passées du groupe d'utilisateurs. « *Les utilisateurs similaires aiment les mêmes objets.* »

Contenu

Basé sur le Contenu

Recommande des objets similaires à ceux déjà appréciés par l'utilisateur en analysant leurs descripteurs/caractéristiques physiques.

Hybride

Systèmes Hybrides

Combine les approches collaboratives et contenu pour compenser le démarrage à froid (*Cold Start*) et la sparsité des données.

Point Clé : Le Filtrage Collaboratif ne nécessite aucune connaissance sur la nature des items, tandis que le Filtrage Basé sur le Contenu nécessite une extraction d'attributs métier détaillée (mots-clés, thématiques, caractéristiques techniques).

4. Filtrage Basé sur le Contenu (Content-Based)

L'objectif est de modéliser le profil utilisateur P_u et le vecteur d'attributs de l'item V_i dans un même espace vectoriel, puis de mesurer leur proximité.

1. Représentation TF-IDF (Texte)

Pour chaque mot t dans un document d :

$$TF\text{-}IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

où $IDF(t, D) = \log(|D| / |\{d \in D : t \in d\}|)$.

2. Score de Similarité Cosinus

Appariement entre le profil P_u et l'item V_i :

$$Sim(P_u, V_i) = (P_u \cdot V_i) / (\|P_u\| \times \|V_i\|)$$

Valeur comprise entre 0 (orthogonal) et 1 (identique).

5. Filtrage Collaboratif (Collaborative Filtering - CF)

Le CF exploite la matrice Utilisateurs-Objets R de taille $M \times N$. Deux approches principales basées sur la mémoire (voisinage) existent :

A. User-Based CF (Orienté Utilisateur)

Cherche les utilisateurs k-NN les plus proches de u .

$$\hat{r}_{u,i} = u_{\text{bar}_u} + \left(\sum_{v \in N(u)} Sim(u,v) \cdot (r_{v,i} - u_{\text{bar}_v}) \right) / \left(\sum |Sim(u,v)| \right)$$

Ajusté par la note moyenne u_{bar_u} de chaque utilisateur pour éliminer le biais de sévérité.

B. Item-Based CF (Orienté Objet)

Recherche les objets similaires à ceux notés par u .

$$\hat{r}_{u,i} = \left(\sum_{j \in N(i)} Sim(i,j) \cdot r_{u,j} \right) / \left(\sum |Sim(i,j)| \right)$$

Généralement plus stable et rapide car le catalogue d'items évolue moins vite que la base utilisateurs.

Mesures de Similarité Fondamentales

Métrique	Formule Mathématique	Cas d'usage optimal
Corrélation de Pearson	$Sim(u,v) = \sum (r_{u,i} - u_{\text{bar}_u})(r_{v,i} - u_{\text{bar}_v}) / (\sigma_u \cdot \sigma_v)$	User-based (Corrige les biais de notation de centrage).
Cosinus Ajusté	$Sim(i,j) = \sum (r_{u,i} - u_{\text{bar}_u})(r_{u,j} - u_{\text{bar}_u}) / (\ i\ _u \cdot \ j\ _u)$	Item-based (Soustrait la moyenne utilisateur par ligne).
Indice de Jaccard	$Sim(A, B) = A \cap B / A \cup B $	Données implicites/binaires (Achat/Non-achat).

Avantages CF

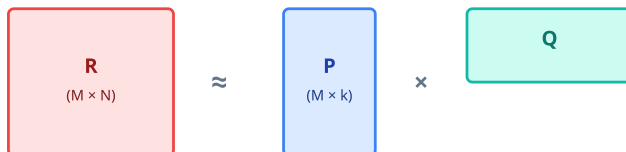
- Indépendant du domaine (pas besoin de tags/metadata).
- Capacité de sérendipité (découverte inattendue).

Inconvénients CF

- Sparsité extrême de la matrice (souvent > 99% vide).
- Cold Start (Nouveau produit / Nouvel utilisateur).

6. Factorisation de Matrices (Matrix Factorization)

Plutôt que d'utiliser des méthodes heuristiques de voisinage, la factorisation de matrice projette simultanément utilisateurs et objets dans un espace de **facteurs latents** de dimension restreinte $k \ll \min(M, N)$.



$P^T (k \times N)$ Signification des Composantes :
 • **P (User Factors)** : Affinité de u pour les k thèmes.
 • **Q (Item Factors)** : Présence des k thèmes dans i .
 • **Prédiction** : $\hat{r}_{u,i} = p_u \cdot q_i^T$

A. Algorithme Funk-SVD avec Biais

Pour capturer les variations individuelles de comportement, on intègre un biais global μ , un biais utilisateur b_u et un biais item b_i :

$$\hat{r}_{u,i} = \mu + b_u + b_i + p_u \cdot q_i^T$$

B. Fonction Objectif (Minimisation de l'Erreur Régularisée)

$$\min_{P, Q, b} \sum_{(u,i) \in R_{\text{obs}}} (r_{u,i} - \hat{r}_{u,i})^2 + \lambda (\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)$$

Le paramètre λ contrôle la régularisation (L2) pour éviter le surapprentissage (*overfitting*) sur les paires observées.

Optimisation : SGD

Stochastic Gradient Descent : Pour chaque note observée, calcul de l'erreur $e_{u,i} = r_{u,i} - \hat{r}_{u,i}$ puis mise à jour rapide :

$$p_u \leftarrow p_u + \gamma(e_{u,i} q_i - \lambda p_u)$$

$$q_i \leftarrow q_i + \gamma(e_{u,i} p_u - \lambda q_i)$$

Optimisation : ALS

Alternating Least Squares : Fixe alternativement **P** pour résoudre quadratiquement **Q**, puis fixe **Q** pour résoudre **P**. Idéal pour la parallélisation distribuée (Spark) et le feedback implicite.

7. Approches Hybrides & Deep Learning

Stratégies d'Hybridation

- **Pondérée (Weighted)** : Combinaison linéaire des scores $S_{\text{hyb}} = \alpha S_{\text{CF}} + (1-\alpha) S_{\text{CB}}$.
- **Cascade** : Un premier modèle génère un Top-100, un second affine le classement.
- **Feature Combination** : Injection des descripteurs de contenu comme dimensions dans le modèle latent.

Réseaux Neuraux (Two-Tower / NeuMF)

Les architectures modernes (ex: YouTube Recommender) utilisent deux tours de réseaux de neurones (User Tower & Item Tower) générant de riches *embeddings* combinés par un produit scalaire ou un MLP final.

8. Évaluation des Métriques de Performance

L'évaluation s'effectue hors-ligne (offline split train/test), en ligne (A/B Testing) ou par étude utilisateurs.

A. Métriques d'Erreur de Prédiction (Rating)

Mesurent la précision de l'estimation de la note numérique.

- **MAE (Mean Absolute Error) :**

$$MAE = (1/T) \sum |r_{\{u,i\}} - \hat{r}_{\{u,i\}}|$$
- **RMSE (Root Mean Squared Error) :**

$$RMSE = \sqrt{(1/T) \sum (r_{\{u,i\}} - \hat{r}_{\{u,i\}})^2}$$
 (Pénalise fortement les grandes erreurs).

B. Métriques de Ordre & Top-K (Ranking)

Mesurent la qualité du classement restitué à l'utilisateur.

- **Precision@K :** Proportion d'items pertinents parmi les K recommandés.
- **Recall@K :** Proportion d'items pertinents recommandés par rapport au total des pertinents.
- **NDCG@K :** Mesure le gain cumulé actualisé (pondère selon le rang d'apparition).

9. Grands Défis Pratiques

Problématique	Description du Phénomène	Techniques de Résolution
Cold Start	Impossibilité d'évaluer un nouvel utilisateur ou produit sans historique.	Basé contenu initial, questionnaires à l'inscription, bandits manchots (MAB).
Sparsité (Matrice Creuse)	Le ratio de cellules remplies dans la matrice R est souvent < 1%.	Factorisation Matricielle (SVD/ALS), Autoencodeurs, Réduction de dimension.
Bulle de Filtre / Biais	Sur-recommandation d'items populaires (Popularity Bias) réduisant la diversité.	Pénalisation de la popularité, optimisation multi-objectifs (Sérendipité / Nouveauté).
Passage à l'Échelle	Calculs impraticables en temps réel pour Millions d'utilisateurs/objets.	Recherche de plus proches voisins approximative (ANN : FAISS, HNSW, LSH).

10. Fiche Récapitulative / Anti-Sèche Examens

Checklist de Choix d'un Modèle RecSys :

1. **Si vous n'avez QUE du texte/metadata et peu d'utilisateurs :** Privilégiez Content-Based (TF-IDF + Cosine).
2. **Si vous avez beaucoup d'interactions historiques et cherchez de la sérendipité :** Privilégiez Collaborative Filtering (Matrix Factorization / SVD).
3. **Si la matrice est géante et distribuée (Spark/Big Data) :** Utilisez ALS (Alternating Least Squares).
4. **Si vous êtes en production temps réel à grande échelle :** Architecture 2-Tiers : 1. *Candidate Generation* (ANN FAISS) → 2. *Heavy Ranking* (MLP / Deep Learning).

